Test Methods

Test Techniques are called as Test methods.

There are mainly 3 types of test techniques:

1. Blackbox or specification based testing:

Testing the software based on requirements without knowing the internal code structure.

Example: For a login page, tester checks whether valid username/password allows login and invalid credentials show an error.

2. Whitebox or clearbox or structural or glassbox testing

Testing the software by understanding the internal code, logic, conditions, loops, and structure..

Example: Developer/tester checks the login function code to verify all if-else conditions for valid user, invalid password, blocked user, and empty fields are tested.

3. Experienced based testing:

Testing based on tester's experience, intuition, domain knowledge, and past defect patterns.

Example: Tester checks if the password field accepts spaces, copy-paste, emojis, very long input, or SQL injection because similar bugs were found before.

Blackbox Testing Methods

Black-box test techniques, also called **specification-based** techniques, focus on the specified behavior of the software, not its internal structure. This means test cases don't depend on how the software is built. If the behavior remains the same, the test cases are still useful even if the implementation changes.

Blackbox has 4 types of test techniques:

1. Equivalence Partitioning
2. Boundary Value Analysis (BVA)
3. Decision Table Testing
4. State Transition Testing

Equivalence Partitioning:

Scenario: A software system accepts birthday of members only from 18 to 100 years.

Invalid	valid	invalid
<18	18-100	>100

Boundary Value Analysis:

2 types of BVA:

1. **2BVA: 17,18,100,101 (min-1, min, max, max+1)**
2. **3BVA: 17,18,19,99,100,101 (min-1, min, min+1, max-1, max, max+1)**

Equivalence Partitioning (Complex scenario):

A bank offers Gold and Premium savings accounts for 1 and 3 years for each plan. The Gold savings account offers 5% interest for 1 year and 7% interest for 3 years, while the Premium savings account offers 7% interest for 1 year and 9% interest for 3 years. To avail of the Gold savings account, one must deposit a minimum of 1 lakh taka, and to avail of the Premium savings account, a deposit of 3 lakh taka is required.

A person deposited 1 lakh taka to purchase a Gold savings account for 3 years, and after 6 months, he deposited an additional 3 lakh taka to purchase a Premium savings account for 1 year.

How many test cases are covered by this scenario, expressed as a percentage?

Decision Table Testing:

Scenario: You have to test a login page.

Conditions	T1	T2	T3	T4
Email	T	T	F	F
Password	T	F	T	F
Output	x		-	N/A

State Transition:

State Transition is a test design technique where we test how a system moves from one state to another based on specific actions, inputs, or events.

Example: In a login system, a user account may move through different states:

Active → Locked → Unlocked

Whitebox Testing Methods

White Box testing technique is generally a testing technique which can be applied by writing test cases based on coding logic.

We can say it **glassbox testing** or **clearbox** testing technique.

Why whitebox testing is important?

- **Understand code logic:** SDET can identify hidden bugs inside conditions, loops, validations, and business rules.
- **Design better test cases:** Code understanding helps cover positive, negative, edge, and branch scenarios more effectively.
- **Improve automation quality:** SDET can write stronger API, integration, and E2E automation by understanding the internal flow.
- **Find root cause faster:** When a test fails, SDET can debug logs, code, database, and API responses instead of only reporting "it failed."
- **Test backend and integration properly:** Many bugs happen between UI, API, database, queue, cache, and third-party services. White-box knowledge helps detect them.
- **Support developers better:** SDET can explain defects with technical evidence, possible root cause, and clear reproduction steps.

- **Reduce production risk:** Internal code-path coverage reduces the chance of hidden logic defects going to production.

Types of whitebox testing:

1. **Statement Testing:** Checks whether each important line or statement of code is executed at least once.
2. **Branch / Decision Testing:** Checks whether each decision point, like if-else, covers both true and false outcomes.
3. **Condition Testing:** Checks individual conditions inside complex logic, especially when multiple conditions are combined.
4. **Unit Testing:** Tests a single function, method, class, or component in isolation to verify its internal logic.
5. **Integration Testing:** Tests how multiple modules, APIs, services, database, or components work together internally.

Experienced based testing techniques

3 types of experienced based testing techniques

1. Error guessing
2. Exploratory testing
3. Checklist-based testing

1. Error Guessing:

Error guessing is a technique used to **anticipate** the occurrence of errors, defects, and failures, based on

the tester's knowledge, including:

1. How the application has worked in the past
2. The types of errors the developers tend to make and the types of defects that result from these errors
3. The types of failures that have occurred in other, similar applications

Testers need to make a list of possible errors, defects, and failures, and then create tests that can find these issues or even make them happen.

Error factors:

Mistakes, issues, and problems in software can be linked to various things like:

1. **Input problems, like not accepting correct input or having the wrong or missing parameters.**
2. **Output issues, such as producing the wrong format or result.**
3. **Logic errors, which could involve missing cases or using the wrong operators.**
4. **Computation problems, like working with the wrong numbers or making incorrect calculations.**
5. **Interface-related troubles, such as parameter mismatches or incompatible types.**
6. **Data issues, including incorrect starting values or using the wrong data types.**

2. Exploratory Testing:

Exploratory testing is a way of testing where the tester **learns, designs, and runs** tests at the same time to understand the test subject better. Often, it's done in a **time-limited session** with specific goals called a **test charter**. After testing, **the tester shares** what they found with others.

The tester explores areas that haven't been tested yet and may use a simple sheet to note down what they did and discovered. Simple notes and findings are essential in this type of testing.

Lightweight documentation are the crucial part of exploratory testing.

3. Checklist based testing techniques:

In **exploratory testing**, testers do not follow detailed step-by-step test cases. They explore the software using their experience, curiosity, and domain knowledge. So a checklist helps them stay focused and avoid missing important areas.

Example checklist for exploratory testing of a login page. Often this is considered as test title.

- Verify that valid users can log in successfully.
- Verify that users cannot log in with an invalid password.
- Verify that proper validation appears when email or password is empty.
- Verify that locked or blocked users cannot log in.
- Verify that the password visibility icon works properly.
- Verify that the remember me option works as expected.

- Verify that the forgot password link redirects users to the correct page.
- Verify that proper error messages appear for invalid login scenarios.
- Verify that session or token behavior works correctly after login.
- Verify that login works properly across different browsers and devices.

Generic checklist for blackbox testing

- Is the end-to-end user flow smooth and uninterrupted?
- Is input validation working correctly (invalid, empty, special characters)?
- Are error messages clear, meaningful, and helpful to users?
- Are buttons and user actions functioning correctly?
- Is the application state preserved correctly on back/refresh?
- Are rapid user actions (e.g., double-click, multiple submissions) handled properly?
- Is frontend (UI) validation consistent with backend validation?
- Does the application work correctly across different devices and browsers?
- Is the UI responsive across mobile, tablet, and desktop?
- Is there any UI flickering, layout breaking, or misalignment?
- Are form submissions idempotent (no duplicate requests on retry)?
- Is session behavior correct (timeout, auto logout, token expiry)?
- Are user actions reflected immediately (UI feedback after click/submit)?
- Is navigation consistent (menu, routing, redirects)?
- Are error scenarios (network failure, API timeout) handled gracefully?

UI testing is not just about clicking buttons — it’s about validating how real users behave, break, retry, and interact under imperfect conditions

Generic checklist for whitebox testing

- Is each important code statement executed by at least one test?
- Are all major if-else or decision outcomes tested properly?
- Are individual conditions inside complex logic tested correctly?
- Are boundary values handled correctly in the code?
- Are null, undefined, empty, and invalid values handled safely?
- Are default and optional parameter values tested properly?
- Are exceptions and error-handling blocks tested correctly?
- Are important business rules implemented correctly in the code logic?
- Are service, controller, repository, API, and database layers interacting correctly?
- Are security checks like authentication, authorization, token validation, and permission checks implemented correctly?

White-box testing is not just about code coverage — it is about verifying internal logic, hidden conditions, business rules, error handling, and system behavior from inside the code.